UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS



PRÁCTICA 05, “SIMULADOR DE GESTOR DE MEMORIA EN C++”

Curso: Sistemas Operativos

Docente: Hugo Manuel Barraza Vizcarra

Autor:

Pedro Czar Paolo Serrano Yanapa / 2024-119023

Fecha: 07 / 10 / 26

TACNA - PERÚ

OBJETIVOS

- Analizar y codificar algoritmos de asignación de memoria
- Comprender las ventajas y desventajas de los algoritmos usados

PROCEDIMIENTO

De acuerdo con lo visto en teoría, existen tres estrategias clásicas de asignación de memoria (en particionamiento dinámico). A continuación se resume su funcionamiento:

- Primer ajuste (First-fit): recorre la memoria desde el inicio y asigna el primer bloque libre cuyo tamaño sea mayor o igual que el requerido por el proceso.
- Mejor ajuste (Best-fit): entre todos los bloques libres capaces de alojar el proceso, elige el más pequeño que sea suficiente, buscando minimizar el espacio sobrante.
- Siguiente ajuste (Next-fit): variante de First-fit. La primera búsqueda comienza en el inicio de la memoria; en solicitudes posteriores, retoma la búsqueda desde la posición donde quedó la última asignación, evitando reiniciar siempre desde el principio.

A continuación, implementaremos un ejemplo del algoritmo de Primer ajuste (First-fit), asumiendo un esquema de particionamiento dinámico de la memoria.

Cree first.cpp y pegue:

Abrir

first.cpp
~/Documentos/04_memoria

```cpp
#include <iostream>
using namespace std;

void firstFit(int blockSize[], int m, int processSize[], int n)
{
  // Arreglo dinámico para guardar el bloque asignado a cada proceso (-1 si no se asigna)
  int* allocation = new int[n];

  for (int i = 0; i < n; ++i)
    allocation[i] = -1;

  // Algoritmo First-Fit
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < m; ++j) {
      if(blockSize[j] >= processSize[i]) {
        allocation[i] = j;              // asigna bloque j al proceso i
        blockSize[j] -= processSize[i];  // reduce espacio libre del bloque
        break;                          // pasa al siguiente proceso
      }
    }
  }
```

```cpp
  // Salida
  cout << "\nNo. Proceso\tTamano Proceso\tNo. Bloque\n";
  for (int i = 0; i < n; ++i) {
    cout << " " << (i + 1) << "\t\t" << processSize[i] << "\t\t";
    if (allocation[i] != -1)
      cout << (allocation[i] + 1);
    else
      cout << "No Asignado";
    cout << "\n";
  }
}

int main()
{
  int blockSize[]   = {100, 500, 200, 300, 600};
  int processSize[] = {212, 417, 112, 301};

  int m = sizeof(blockSize)   / sizeof(blockSize[0]);
  int n = sizeof(processSize) / sizeof(processSize[0]);

  firstFit(blockSize, m, processSize, n);
  return 0;
}
```

CUESTIONES

- ¿Qué sucede con el proceso 4 definido en processSize con tamaño 301? ¿A qué se debe ese comportamiento?

```
qpa@qpa-VirtualBox:~/Documentos/04_memoria$ g++ first.cpp -o first
qpa@qpa-VirtualBox:~/Documentos/04_memoria$ ./first

No. Proceso     Tamano Proceso  No. Bloque
 1              212             2
 2              417             5
 3              112             2
 4              301             No Asignado
```

- El proceso 4, su No. de bloque no está definido; se debe a que no existe memoria que pueda guardar el tamaño del proceso 4:
  Proceso 4 : 301 tamaño

  Bloque 1: 100 memoria
  Bloque 2: 500 memoria : Proceso 1 y 3 ; Memoria sobrante : 176
  Bloque 3: 200 memoria
  Bloque 4: 300 memoria
  Bloque 5: 600 memoria : Proceso 2 ; Memoria sobrante : 183

  Como se puede apreciar, el proceso 4 no tiene algún bloque en el que la memoria pueda ingresar.

- ¿Qué pasaría si se aplicara el mismo algoritmo, pero a bloques del mismo tamaño vistos en particionamiento fijo de memoria?

  - Si se particionaria los bloques en un mismo tamaño fijo de memoria (la cual la memoria total sería de 1700), cada bloque tendría 340 de memoria.

```
qpa@qpa-VirtualBox:~/Documentos/04_memoria$ g++ first.cpp -o first
qpa@qpa-VirtualBox:~/Documentos/04_memoria$ ./first

No. Proceso     Tamano Proceso  No. Bloque
 1              212             1
 2              417             No Asignado
 3              112             1
 4              301             2
qpa@qpa-VirtualBox:~/Documentos/04_memoria$
```

  Por lo que el proceso 2 de tamaño 417 no podrá ingresar en alguno de los bloques de 340 de memoria cada uno.

ACTIVIDADES PROPUESTAS

- Codificar en C++, un programa para el método Mejor Ajuste y otro programa para Próximo Ajuste. Para ello en el informe debe indicar ejemplos numéricos con su resultado esperado para probar que funcionen ambos métodos.
- Ambos métodos deben ser hechos tomando en cuenta el particionamiento dinámico de los bloques de memoria.

Mejor Ajuste:

```cpp
#include <iostream>
using namespace std;

void firstFit(int blockSize[], int m, int processSize[], int n)
{
  // Arreglo dinámico para guardar el bloque asignado a cada proceso (-1 si no se asigna)
  int* allocation = new int[n];
  int blockOpens[m];
  int blockMin;
  int k = 0;

  for (int i = 0; i < n; ++i)
    allocation[i] = -1;
```

```cpp
  // Algoritmo Best-Fit
  for (int i = 0; i < n; ++i) {
    k = 0;
    for (int j = 0; j < m; ++j) {
      if(blockSize[j] >= processSize[i]) {
        blockOpens[k] = blockSize[j];   // Busca los bloques compatibles para el proceso
        k = k + 1;
      }
    }
    blockMin = blockOpens[0];
    for (int l = 1; l < k; ++l){
      if (blockMin >= blockOpens[l]){
        blockMin = blockOpens[l];       // Verifica cual bloque compatible es el mas corto
      }
    }
    for (int o = 0; o < m; ++o){
      if (blockMin == blockSize[o]){
        allocation[i] = o;              // Asigna bloque o al proceso i
        blockSize[o] -= processSize[i]; // Reduce el bloque o
        break;
      }
    }
  }
```

```
  // Salida
  cout << "\nNo. Proceso\tTamano Proceso\tNo. Bloque\n";
  for (int i = 0; i < n; ++i) {
    cout << " " << (i + 1) << "\t\t" << processSize[i] << "\t\t";
    if (allocation[i] != -1)
      cout << (allocation[i] + 1);
    else
      cout << "No Asignado";
    cout << "\n";
  }
}

int main()
{
  int blockSize[]   = {100, 500, 200, 300, 600};
  int processSize[] = {212, 417, 112, 301};

  int m = sizeof(blockSize)   / sizeof(blockSize[0]);
  int n = sizeof(processSize) / sizeof(processSize[0]);

  firstFit(blockSize, m, processSize, n);
  return 0;
}
```

Próximo ajuste:

```cpp
#include <iostream>
using namespace std;

void firstFit(int blockSize[], int m, int processSize[], int n)
{
  // Arreglo dinámico para guardar el bloque asignado a cada proceso (-1 si no se asigna)
  int* allocation = new int[n];
  int punt, j, l;

  for (int i = 0; i < n; ++i)
    allocation[i] = -1;

  // Algoritmo Next-Fit
  for (int i = 0; i < n; ++i) {
    j = punt;
    l = 0;
    while (l != 2){
      if(j > m){                        // cuando j pasa de los limites del arreglo, vuele a 0
        j = 0;
        l += 1;                         // nuevo valor l para no repetir indefinidamente si no hay memoria
      }
      if(blockSize[j] >= processSize[i]) {
        allocation[i] = j;              // asigna bloque j al proceso i
        blockSize[j] -= processSize[i]; // reduce espacio libre del bloque
        punt = j;                       // asigna el puntero posterior de la ejecución
        break;                          // pasa al siguiente proceso
      }
      j = j + 1;
    }
  }
```

```
  // Salida
  cout << "\nNo. Proceso\tTamano Proceso\tNo. Bloque\n";
  for (int i = 0; i < n; ++i) {
    cout << " " << (i + 1) << "\t\t" << processSize[i] << "\t\t";
    if (allocation[i] != -1)
      cout << (allocation[i] + 1);
    else
      cout << "No Asignado";
    cout << "\n";
  }
}

int main()
{
  int blockSize[]   = {100, 500, 200, 300, 600};
  int processSize[] = {212, 417, 112, 301};

  int m = sizeof(blockSize)   / sizeof(blockSize[0]);
  int n = sizeof(processSize) / sizeof(processSize[0]);

  firstFit(blockSize, m, processSize, n);
  return 0;
}
```

- Pushear los archivos junto con el informe a su repositorio, para probar su funcionamiento.

Mejor Ajuste:

Procesos:

- Proceso 01: Tamaño 212
- Proceso 02: Tamaño 417
- Proceso 03: Tamaño 112
- Proceso 04: Tamaño 301

Bloques / Memoria:

- Bloque 1: 100
- Bloque 2: 500
- Bloque 3: 200
- Bloque 4: 300
- Bloque 5: 600

Ejecución:

```
qpa@qpa-VirtualBox:~/Documentos/04_memoria$ ./best

No. Proceso     Tamano Proceso  No. Bloque
 1              212             4
 2              417             2
 3              112             3
 4              301             5
qpa@qpa-VirtualBox:~/Documentos/04_memoria$
```

Próximo Ajuste:

Procesos:

- Proceso 01: Tamaño 212
- Proceso 02: Tamaño 417
- Proceso 03: Tamaño 112
- Proceso 04: Tamaño 301

Bloques / Memoria:

- Bloque 1: 100
- Bloque 2: 500
- Bloque 3: 200
- Bloque 4: 300
- Bloque 5: 600

Ejecución:

```
qpa@qpa-VirtualBox:~/Documentos/04_memoria$ g++ next.cpp -o next
qpa@qpa-VirtualBox:~/Documentos/04_memoria$ ./next

No. Proceso     Tamano Proceso  No. Bloque
 1              212             2
 2              417             5
 3              112             5
 4              301             No Asignado
qpa@qpa-VirtualBox:~/Documentos/04_memoria$
```